

Basketball Draft Predictor

Matthew Arboleda
Occidental College
arboleda@oxy.edu

Kincaid Fries
Occidental College
kfries@oxy.edu

Ethan Schoen
Occidental College
eschoen@oxy.edu

Abstract

Forecasting a college basketball player’s NBA career outcome is a high-stakes prediction problem that blends structured performance data with unstructured qualitative assessments. We present a multi-pipeline framework aimed ultimately at *multimodal* prediction: (1) regularized linear models and gradient-boosted trees on engineered NCAA per-game and efficiency statistics; (2) a fine-tuned DistilBERT encoder and a shallow lexicon / sentiment baseline on pre-draft scouting text; and (3) a logistic stacker fusing tier probabilities from (1) and (2). We evaluate continuous regression on a normalized NBA role-score target and four-class ordinal tiers (Bust / Bench / Starter / Star). Tabular XGBoost is the strongest single predictor—roughly $R^2 \approx 0.29$ on role score and 42.4% tier accuracy with macro-F1 0.437—while linear models lag and Ridge can fail badly under multicollinearity; standalone text models trail tabular performance yet still encode usable tier distributions for fusion. The stacking ensemble yields our best aggregate outcome: 43.8% tier accuracy, quadratic weighted $\kappa=0.490$, and 84.2% of prospects within one ordinal tier of the label, clearly above any individual base model and improving ordinal agreement over the best tree classifier alone. All experiments are tracked with MLflow under a reproducible uv-managed environment.

1 Introduction

The NBA Draft represents a pivotal moment for franchises, organizations, and players. Teams invest millions and countless man-hours within scouting infrastructure, yet bust rates remain stubbornly high: around 40% of first-round picks fail to have a career longer than three years [6]. Accurate pre-draft outcome prediction would help teams improve their rosters, reduce wasted salary-cap commitments, and give college programs better signals for what is truly valued in one of the most talented leagues in the world.

Prior analytical work has mainly treated draft evaluations as a tabular regression problem, using box-score features to fit a linear or tree-based model and evaluate on held-out draft classes [3]. Within this work, two data sources are rarely used together: qualitative assessments written by professional scouts, and temporal context encoded across multiple college seasons. Modern pre-trained language models and multimodal deep learning offer principled ways to incorporate both signals.

2 Related Work

2.1 Statistical Models for Draft Evaluation

Kubatko et al. [3] introduced Win Shares as a composite efficiency metric and showed how college performance correlates meaningfully with early NBA production. Oliver [4] formalized the “Four Factors”: shooting efficiency, turnover rate, rebounding, and free-throw rate. We use these features as `fg_pct`, `ft_rate`, `reb_pg`, and `pts_per_fga` within our pipeline.

2.2 Career-Longevity Prediction

Staw and Hoang [6] studied the sunk-cost effect in NBA roster decisions, motivating career length as an outcome metric independent of on-court efficiency. Alamar [1] notes that binary survival labels are often better calibrated than continuous value metrics for early-career prospects whose performance is highly noisy.

2.3 NLP and Scouting Reports

Text-based player evaluation has very limited formal study. The success of BERT-family models on domain-specific fine-tuning [2] and the computational efficiency of DistilBERT [5] motivate our backbone choice, balancing expressiveness with inference cost.

3 Methods

3.1 Dataset Construction

We construct the dataset by joining three sources using `src/data/loader.py`.

NCAA statistics. Season-level box-score totals stored in `data/ncaa/ncaa_master.csv` are joined to NBA outcomes on (`player_name`, `draft_year`).

NBA career outcomes. From `data/nba/nba_master.csv` (joined on (`player_name`, `draft_year`)) we derive two aligned targets from one NBA career composite. For each of minutes, points, rebounds, assists, steals, and blocks we compute a z-score—globally across players by default,

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org. *Conference’17, Washington, DC, USA*

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

or within NBA position group when configured—then win-size at ± 2.5 . A weighted average combines these clipped z-scores; the composite is re-standardized to unit variance as `nba_role_zscore`. **Regression** predicts `nba_role_zscore` directly. **Classification** predicts `prospect_tier`: fixed thresholds on the same score (defaults $(-0.5, 0.5, 1.5)$ on the standardized composite) bin prospects into four ordinal tiers—*Bust*, *Bench*, *Starter*, *Star*—encoded as integer labels 0–3.

Scouting reports. Pre-draft evaluations in `data/scouting/players.csv` are joined on (`player_name`, `draft_year`) after normalizing names (stripping leading draft-rank numbers via regex). Only rows with non-empty `full_scouting_report` text are retained for the text pipeline.

Data splits. Tabular models use chronological splits by draft year; multimodal training uses the same chronology, and we evaluate the final stacker on stratified random train/validation/test. The text model uses shuffled `train_test_split` over all draft classes (fixed `random_state=42`; default 70%/15% train/val, remainder test).

3.2 Feature Engineering

The following features are derived from raw totals in `load_data()`:

Per-game rates. All counting stats are divided by games played G (zero replaced by NaN, backfilled from pre-computed columns where available): `pts_pg`, `reb_pg`, `ast_pg`, `blk_pg`, `stl_pg`, `fgm_pg`, `fga_pg`, `ft_pg`, `fta_pg`, `fg3_pg`.

Efficiency ratios:

$$\begin{aligned} \text{fg_pct} &= \frac{\text{FGM}}{\text{FGA}} \times 100, & \text{pts_per_fga} &= \frac{\text{PTS}}{\text{FGA}}, \\ \text{ft_rate} &= \frac{\text{FTA}}{\text{FGA}}, & \text{fg3_share} &= \frac{\text{3FG}}{\text{FGM}}. \end{aligned} \quad (1)$$

Categorical / ordinal features. Player position (`Pos`) is one-hot encoded; academic class (`Cl`: Fr., So., Jr., Sr.) is ordinally encoded and standardized. Height is parsed from a “feet-inches” string to total inches. Conference tier (`conf_tier`) is assigned as 2 (Power 5), 1 (high mid-major), or 0 (other) based on a curated school list.

A shared `ColumnTransformer` applies median imputation and standard scaling to numeric/ordinal columns, and mode imputation with one-hot encoding to categorical columns. Draft-pick rank (`draft_pick`) is optional (`use_draft_pick`, default `False`): values follow <https://nbadraft.net> pre-draft projection/ranking, not actual NBA draft position.

3.3 Tabular Baselines

Lasso (LassoCV): L_1 -regularized least squares; α selected by 5-fold CV over 100 log-spaced values in $[10^{-3}, 10^2]$, `max_iter=10000`. L_1 regularization induces sparsity and automatic feature selection.

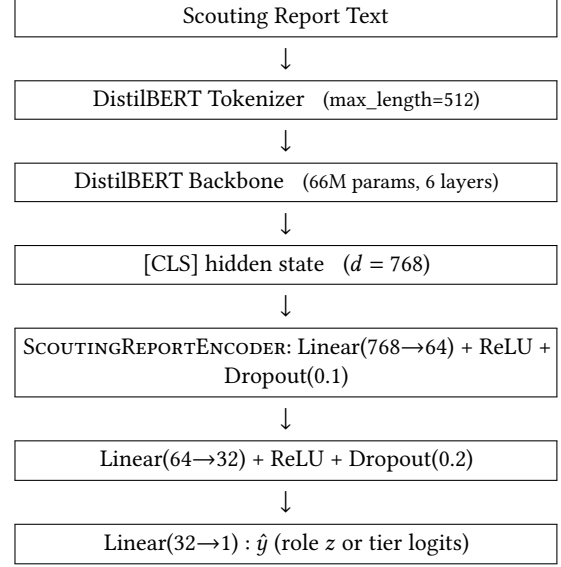


Figure 1. TEXTPROSPECTPREDICTOR architecture. DistilBERT [5] with SCOUTINGREPORTENCODER (768→64) and head; predicts role z or tier logits.

Ridge (RidgeCV): L_2 -regularized least squares; same α grid and fold count. Ridge retains all features with shrinkage, complementing Lasso for features that are collectively but not individually informative.

XGBoost (XGBRegressor / XGBClassifier): gradient-boosted ensemble with 3-fold grid search over $n_{\text{est}} \in \{100, 200\}$, `max_depth` $\in \{2, 3\}$, `lr` $\in \{0.05, 0.1\}$, `subsample` $\in \{0.7, 0.9\}$. `min_child_weight=5` is fixed for regularization. Selection criterion: cross-validated R^2 (regression) or macro-F1 for four-tier classification (`prospect_tier`).

Logistic variants (classification only): `LogisticRegressionCV` with the `saga` solver, 20 log-spaced C values in $[10^{-3}, 10^2]$, 5-fold CV, for both L_1 (LOGISTICL1) and L_2 (LOGISTICL2) penalties.

All models are wrapped in `sklearn Pipelines` to prevent preprocessing leakage.

3.4 Text Model: ScoutingReportEncoder

The text pipeline fine-tunes `distilbert-base-uncased` [5] (Figure 1). We additionally train a second, *narrow-capacity* text path (VADER sentiment, lexicon counts, Ridge) alongside DistilBERT; both approaches contribute tier probabilities to multimodal stacking.

ScoutingReportEncoder. Pools [CLS] ($d = 768$) through `Linear(768→64) + ReLU + Dropout(0.1)`. Backbone weights can optionally be frozen (`freeze_base=True`).

TextProspectPredictor. `Linear(64→32) + ReLU + Dropout(0.2)`, then one regression output or `num_classes` logits.

Training details. AdamW optimizer (`lr` $= 10^{-3}$, weight decay 10^{-4}), batch size 32, Smooth L1 loss ($\beta = 1$), gradient

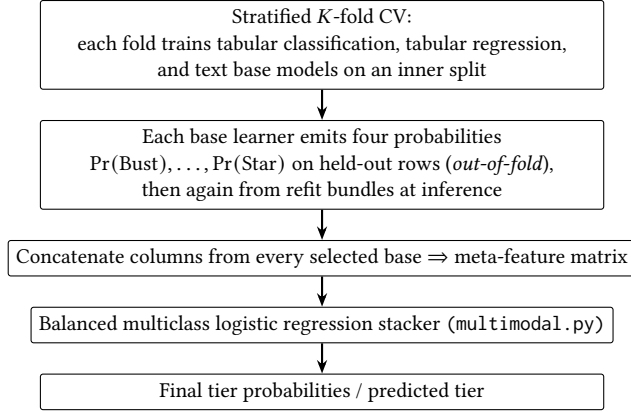


Figure 2. Probability stacking evaluated in Section 4.4. Train rows receive *out-of-fold* tier probabilities from each base family so the stacker does not train on its own inputs. After stacking weights are fit, final base models are retrained on the full training split for deployment.

clipping at 1.0, up to 50 epochs with early stopping (patience 10); best checkpoint by validation loss. Device is auto-selected: CUDA > MPS > CPU.

3.5 Multimodal Ensemble (Probability Stacking)

As illustrated in Figure 2, MULTIMODALPROSPECTMODEL trains configurable tabular classification bundles, tabular regression bundles (Gaussian tier probabilities exported from predicted `nba_role_zscore`), and one or more text bundles on stratified folds. For each fold, base models fit on an inner training subset and emit four-way tier probabilities on held-out rows; those columns form the supervised inputs to a balanced multiclass logistic regression *stacker*. At inference, final base bundles produce the same probability columns on fresh rows and the stacker outputs aggregated tier probabilities.

3.6 Experimental Conditions

We compare five experimental conditions, each logged as a distinct MLflow parent run: (1) regression without draft pick (target `nba_role_zscore`); (2) regression with draft pick; (3) classification on `prospect_tier`; (4) text model with full fine-tuning (`freeze_base=False`); (5) text model with frozen backbone (`freeze_base=True`).

3.7 Evaluation Metrics

Regression (`nba_role_zscore`): R^2 , RMSE, MAE.

Tier classification (`prospect_tier`): accuracy; balanced accuracy; macro-averaged precision, recall, and F1; quadratic weighted κ (QWK) between predicted and true tier indices, penalizing larger ordinal mistakes more than nearer misses.

3.8 Infrastructure and Reproducibility

All runs use MLflow’s nested parent-child structure: one parent run per model family, one child run per estimator. Hyperparameter configurations, a candidate-summary JSON, and fitted model artifacts are logged automatically. Local PNG plots are written to `outputs/plots/{run_name}/`. The full environment is specified by `pyproject.toml` and a uv lock-file; reproduction requires three shell commands: `git clone`, `uv sync`, and `uv run python src/main.py`.

4 Results

4.1 Regression

Table 1 reports held-out performance for predicting `nba_role_zscore`. XGBoost is the clear winner, explaining roughly 29% of variance in career role score. Lasso recovers only 5% of variance, retaining just three features (RPG, BPG, FG%); Ridge collapses entirely ($R^2 = -1.56$), indicating its uniform shrinkage is poorly suited to the high multicollinearity in this feature set.

Table 1. Regression results on held-out set (`nba_role_zscore`).

Model	R^2	RMSE	MAE
Lasso	0.048	0.981	0.761
Ridge	-1.562	1.609	0.841
XGBoost	0.293	0.845	0.704

4.2 Classification

Table 2 reports 4-class tier prediction (Bust / Bench / Starter / Star). XGBoost again leads across all metrics, achieving 42.4% accuracy and a macro-F1 of 0.437. Both logistic variants plateau at identical accuracy (32.3%), suggesting a linear decision boundary cannot separate the tier space. Balanced accuracy of 0.452 for XGBoost indicates better handling of the class imbalance introduced by the Star tier.

Table 2. 4-class classification results (Bust / Bench / Starter / Star).

Model	Accuracy	F1-macro	Bal. Acc
Logistic L_1	0.323	0.315	0.353
Logistic L_2	0.323	0.316	0.355
XGBoost	0.424	0.437	0.452

4.3 Text Models

We evaluate two text modeling approaches on the same regression and classification targets; both appear as text towers in the multimodal stacker (Table 3).

Version 1 (DistilBERT fine-tuned). Regression MAE ≈ 0.92 ; classification accuracy ≈ 0.29 , macro-F1 ≈ 0.17 . The

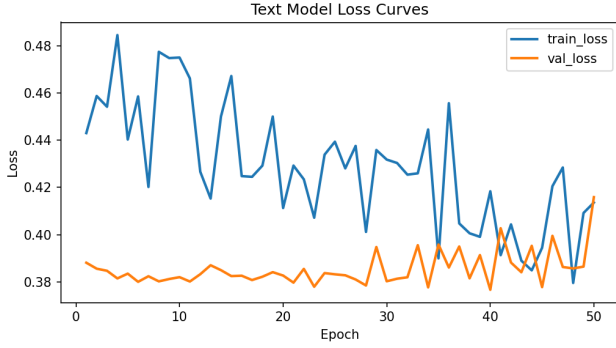


Figure 3. Training and validation loss curves for DistilBERT fine-tuned model.

model converges to predicting near-zero z-scores for most players (Figure 1), reflecting the difficulty of grounding scouting language to a noisy continuous outcome with fewer than 800 labeled examples.

Version 2 (VADER + lexicon features + Ridge). Regression MAE ≈ 0.90 , $R^2 \approx 0.00$. The shallow model achieves marginally lower MAE than the fine-tuned transformer with far fewer parameters (Table 3, Lexicon), suggesting comparable usable signal from handcrafted features in this data regime.

4.4 Multimodal Meta-Model

Table 3 compares the stacking meta-model against each base model. The stacker improves accuracy by 2.4 percentage points over the best single base model (CLF-XGBoost, 0.414) and raises macro-F1 to 0.431. Quadratic weighted kappa (QWK) increases to 0.490, substantially above any individual model, indicating that the meta-model learns to penalize large tier errors more effectively. Notably, 84.2% of predictions are within one tier of the true label (0 or 1 tiers off).

Table 3. Multimodal meta-model vs. individual base models.

Model	Acc	Within 1	Dist Wt Acc	Ord MAE	QWK	F1-macro
Stacker (final)	0.438	0.841	0.753	0.740	0.490	0.431
CLF - L_2 Logistic	0.356	0.755	0.699	0.904	0.182	0.344
CLF - XGBoost	0.414	0.789	0.728	0.817	0.200	0.329
REG - Ridge	0.385	0.928	0.771	0.688	0.380	0.283
REG - XGBoost	0.385	0.952	0.779	0.664	0.454	0.305
Text (DistilBERT)	0.380	0.909	0.763	0.712	0.181	0.217
Text (Lexicon)	0.399	0.865	0.752	0.745	0.220	0.263

The confusion matrix for the stacker (Figure 4) shows strong performance on the extremes: 62% of Busts and 82% of Stars are correctly predicted. The Bench and Starter tiers remain the hardest to distinguish (23% and 33% on-diagonal). Figure 5 summarizes errors: 43.8% exact, 40.4% one tier off, 13.9% two tiers off, and 1.9% three tiers off—far better than uniform random.

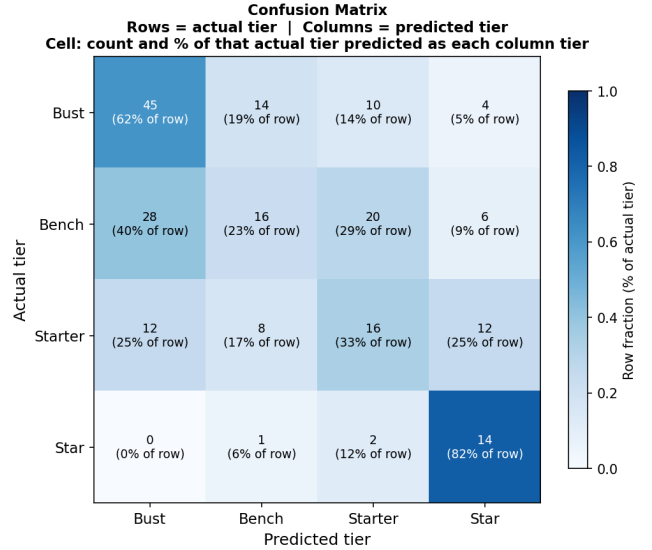


Figure 4. Stacker tier confusion matrix (rows normalized).

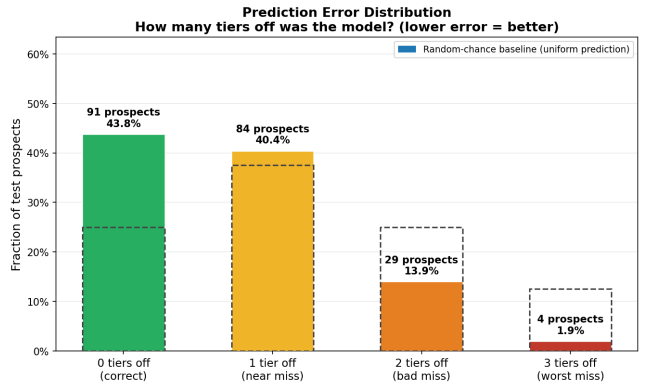


Figure 5. Ordinal tier error (0/1/2/3 tiers off).

5 Discussion

5.1 What Worked

XGBoost consistently outperformed linear models on both regression and classification, confirming that NBA career outcomes are not linearly separable from NCAA statistics. Non-linear feature interactions like the combination of high shooting efficiency and high rebounding rate carry signal that L_1/L_2 regularized models can't exploit. The stacking meta-model raises QWK to 0.490 compared with 0.454 for REG-XGBoost and 0.200 for CLF-XGBoost alone, while improving tier accuracy over the best single classifier. The meta-model's strong performance at the tier extremes (Bust and Star) is practically useful, as these are precisely the decisions (avoiding catastrophic busts and correctly identifying franchise talents) that would matter the most to NBA front offices.

5.2 What Did Not Work

Ridge regression performed catastrophically ($R^2 = -1.56$), likely because uniform weight shrinkage interacts poorly with multicollinearity across highly correlated per-game statistics, producing unstable coefficient estimates. Both text pipelines achieved near-zero R^2 in regression mode. The DistilBERT model collapses to predicting near-zero z-scores for virtually all players (Figure 1), a mode-seeking failure common when training data is scarce relative to model capacity. The shallow VADER+lexicon model performs no better which might suggest that the bottleneck is not model expressiveness but label noise and inconsistency in scouting language itself. The reports vary a lot in length, style, and level of detail across sources and draft years.

5.3 Limitations

Several structural limitations bound the results. First, the feature set captures only on-court production; latent factors like injury history, work ethic, and team fit are not present. Secondly, raw counting statistics are position confounded: a center’s 10 RPG and a guard’s 10 RPG signal different things, yet the shared scalar treats them symmetrically. Third, the four-tier target was defined by z-score thresholds applied to a weighted composite—an arbitrary design choice that affects both class boundaries and class imbalance. The Star tier is severely underrepresented (~10% of players), causing all models to struggle there despite class weighting. Finally, ordinal structure is ignored by softmax classifiers: a Starter predicted as Bust is penalized identically to a Bench predicted as Bust, even though the former is a far worse error.

6 Conclusion

We presented a multi-pipeline framework for NBA draft outcome prediction that spans tabular regression, four class tier classification, NLP-based scouting report analysis, and a stacking meta model. Our main findings are:

1. **Box-score statistics carry the strongest signal.** XGBoost on NCAA per-game and efficiency features achieves $R^2 = 0.293$ in role-score regression and 42.4% accuracy (F1-macro 0.437) in tier classification—substantially above linear baselines.
2. **Scouting text provides complementary but weak signal.** Both the fine-tuned DistilBERT and the shallow lexicon model achieve near-zero R^2 as standalone predictors, yet including them in the stacking ensemble raises QWK from 0.454 to 0.490.
3. **Multimodal stacking improves robustness.** The final meta-model places 84.2% of prospects within one tier of their true label and correctly identifies 82% of eventual Stars—results that suggest practical utility for pre-draft screening.

Future work should address the key limitations identified above. Position-relative feature normalization and draft-class

era adjustment would reduce systematic bias in the tabular features. Ordinal regression or ordinal cross-entropy loss would better respect the rank structure of the tiers. NBA combine data measurements (wingspan, vertical jump, sprint speed) would serve as a principled proxy for the athleticism that scouting reports describe qualitatively but models can’t currently quantify. Finally, a larger and more consistent scouting report corpus, ideally normalized across multiple draft services, could unlock the full representational capacity of the DistilBERT backbone.

References

- [1] Benjamin C. Alamar. 2013. *Sports Analytics: A Guide for Coaches, Managers, and Other Decision Makers*. Columbia University Press.
- [2] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of NAACL-HLT*. 4171–4186.
- [3] Justin Kubatko, Dean Oliver, Kevin Pelton, and Dan Rosenbaum. 2007. A Starting Point for Analyzing Basketball Statistics. *Journal of Quantitative Analysis in Sports* 3, 3 (2007).
- [4] Dean Oliver. 2004. *Basketball on Paper: Rules and Tools for Performance Analysis*. Brassey’s.
- [5] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. 2019. DistilBERT, a Distilled Version of BERT: Smaller, Faster, Cheaper and Lighter. *arXiv preprint arXiv:1910.01108* (2019).
- [6] Barry M. Staw and Ha Hoang. 1995. Sunk Costs in the NBA: Why Draft Order Affects Playing Time and Survival in Professional Basketball. *Administrative Science Quarterly* 40, 3 (1995), 474–494.